

NumPy

📖 **NumPy (Numerical Python)** is a fundamental library for numerical computing in Python.

Provides:

- Fast array operations
 - Mathematical functions
 - Linear algebra support
-

NumPy ndarray (N-Dimensional Array)

The core feature of NumPy is the **ndarray** object. Unlike Python lists, ndarrays are stored in contiguous memory blocks, making them significantly faster for mathematical operations.

- **Homogeneity:** All elements in a NumPy array must be of the same data type (`dtype`).
- **Key Attributes:**
 - `shape`: Returns a tuple representing the dimensions (e.g., `(3, 3)`).
 - `ndim`: The number of axes (dimensions).
 - `size`: Total number of elements.
 - `dtype`: The data type of the elements.

```
import numpy as np
# 1D array
a = np.array([1, 2, 3, 4])
# 2D array
b = np.array([[1, 2],
              [3, 4]])

print(a)
print(b)

print(a.shape) # (4,)
print(b.shape) # (2,2)
print(a.ndim)  # 1
print(b.ndim)  # 2
print(a.dtype) # data type
```

Vectorization Operation

Vectorization is the process of performing operations on entire arrays without using explicit `for` loops.

Batch Processing: Any arithmetic operation between equal-size arrays applies the operation element-wise.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

- Vectorized addition

```
c = a + b
print(c) # [5 7 9]
```

- Scalar Operation

```
print(a * 2)
```

- `import numpy as np`
`a = np.array([1, 4, 9, 16])`
`print(np.sqrt(a))`

- `a = np.array([10, 20, 30, 40])`
`print(a > 25)`

✓ Output:

```
[False False True True]
```

- `a = np.array([1, 2, 3])`
`result = 2*a + 5`
`print(result)`

✓ Output:

```
[7 9 11]
```

Without Vectorization (Slow)

```
result = []
for i in range(len(a)):
    result.append(a[i] + b[i])
```

Broadcasting:

This allows NumPy to work with arrays of different shapes during arithmetic operations. For example, adding a scalar to a matrix:

```
import numpy as np

a = np.array([1, 2, 3])

result = a + 5
print(result)
```

👉 Output:

```
[6 7 8]
```

✓ Explanation:

- Scalar 5 is broadcasted to [5, 5, 5]

1D + 2D Array

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

b = np.array([10, 20, 30])

result = a + b
print(result)
```

👉 Output:

```
[[11 22 33]
 [14 25 36]]
```

👉 b is broadcasted to:

```
[[10 20 30]
 [10 20 30]]
```

Column Broadcasting

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

b = np.array([[10],
```

```
[20]])
```

```
result = a + b  
print(result)
```

👉 Output:

```
[[11 12 13]  
 [24 25 26]]
```

b becomes:

```
[[10 10 10]  
 [20 20 20]]
```

Array Indexing and Slicing

NumPy indexing allows you to select specific elements or sub-sections of an array.

- **One-Dimensional:** Similar to Python lists:

```
a = np.array([10, 20, 30, 40])
print(a[0]) # 10
print(a[-1]) # 40
```

Slicing

```
print(a[1:3])    # [20 30]
print(a[:2])     # [10 20]
print(a[2:])     # [30 40]

print(a[::-1])   # [50 40 30 20 10]
```

a[start : stop : step]

- start → default = beginning (index 0)
- stop → default = end of array
- step → how many positions to skip

```
print(a[::2])    # [10 30]
```

2D Array Indexing

```
b = np.array([[1,2,3], [4,5,6]])
```

Structure of Array

	Column →	0	1	2
Row				
0		[1	2	3]
1		[4	5	6]

```
print(b[0,1]) # 2
```

```
b[row_index, column_index]
```

Access Entire Row

- `print(b[1,:])` # [4 5 6]

Access Entire Column

- `print(b[:,1])` # column -> [2 5]

: means “everything in that axis”

`b[all_rows, column_index]`

Slicing Rows

```
0           [1  2  3]
1           [4  5  6]
print(a[0:2, :])
```

✓ Output:

```
[[1 2 3]
 [4 5 6]]
```

👉 First 2 rows

Slicing Columns

```
print(a[:, 0:2])
```

✓ Output:

```
[[1 2]
 [4 5]]
```

👉 First 2 columns

More examples:

```
import numpy as np

marks = np.array([
    [85, 78, 92], # Student 1
    [88, 76, 95], # Student 2
    [90, 82, 88]  # Student 3
])
```

- 👉 Rows = Students
- 👉 Columns = Subjects

✓ Example Operations

```
print(marks[:,1])    # Subject 2 marks
print(marks[1,:])    # Student 2 marks
```

Shop Sales Data (Broadcasting)

```
sales = np.array([
    [100, 200, 300],
    [150, 250, 350]
])

discount = np.array([10, 20, 30])

print(sales - discount)
```

- 👉 Discount applied to all shops automatically (broadcasting)

4. Transposing Arrays and Swapping Axes

Transposing is a special form of reshaping that returns a view of the data without copying it.

- **.T attribute:** The simplest way to transpose a 2D array (swaps rows and columns).
- **transpose() method:** Can accept a tuple of axis numbers to permute the axes for higher-dimensional arrays.

```
a = np.array([[1,2],
               [3,4]])
```

```
print(a.T)
```

Output:

```
[[1 3]
 [2 4]]
```

Rows become columns and columns become rows

transpose () Method (More Flexible)

Same as .T (2D case)

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(np.transpose(a))
```

✓ Output (same as .T):

```
[[1 4]
 [2 5]
 [3 6]]
```

- **swapaxes () method:** Takes two axis numbers and swaps them.

Swap Axes

```
a = np.array([[[1,2],[3,4]],
              [[5,6],[7,8]])

print(np.swapaxes(a, 0, 1))
```

👉 Changes dimensions (axes)

Meaning of Axes

Axis Meaning

Axis 0 Blocks

Axis 1 Rows

Axis 2 Columns

What does `swapaxes(a, 0, 1)` mean?

👉 Swap:

- Axis 0 (blocks)
- Axis 1 (rows)

- **Before:**

- Open Box 0 → get all rows
Open Box 1 → get all rows

- **After swap:**

- Take Row 0 from all boxes → make new box
Take Row 1 from all boxes → make new box

`[1,2],`

`[3,4]],`

`[[5,6],`

`[7,8]]]`

Output:

`[[[1 2]`
`[5 6]]`

`[[3 4]`
`[7 8]]]`

Simple 2D Case

```
a = np.array([[1, 2, 3],  
             [4, 5, 6]])
```

```
print(np.swapaxes(a, 0, 1))
```

✓ Output:

```
[[1 4]  
 [2 5]  
 [3 6]]
```

👉 Same as transpose

👉 Axis 0 ↔ Axis 1 (rows ↔ columns)

5. Saving and Loading Arrays

NumPy provides efficient methods for storing data on a disk in binary or text formats.

- **Binary Format:**
 - `np.save('filename', array)`: Saves a single array to a `.npy` file.
 - `np.load('filename.npy')`: Reads the array back.
 - `np.savez()`: Saves multiple arrays into an uncompressed `.npz` archive.
- **Text Format:** * `np.savetxt()` and `np.loadtxt()` are used for CSV or other delimited text files.

6. Universal Functions (ufuncs)

A **ufunc** is a **vectorized function** that operates **element-wise** on **NumPy arrays**.

👉 In simple words:

Instead of writing loops, ufuncs automatically apply operations to **each element of an array efficiently**.

💠 Why ufuncs are important?

- ⚡ **Faster execution** (implemented in C internally)
- 🔄 **No need for loops**
- 📊 **Works on arrays of any dimension**

 Supports **broadcasting**

- **Unary ufuncs (Operates on 1D array)**

```
import numpy as np

a = np.array([1, 4, 9, 16])

print(np.sqrt(a)) # Square root
print(np.exp(a)) # Exponential
print(np.sin(a)) # Sine values
```

- **Binary ufuncs: (operate on two arrays)**

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.add(a, b)) # Addition
print(np.subtract(a, b)) # Subtraction
print(np.multiply(a, b)) # Multiplication
print(np.divide(a, b)) # Division
```

Common Examples of ufuncs

Category	Functions
-----	-----
Arithmetic	`add()`, `subtract()`, `multiply()`, `divide()`
Trigonometric	`sin()`, `cos()`, `tan()`
Exponential	`exp()`, `log()`, `sqrt()`
Comparison	`greater()`, `less()`, `equal()`

7. Mathematical and Statistical Functions

In NumPy, **mathematical and statistical functions** are used to perform fast computations on arrays. These functions are mostly implemented as **vectorized operations (ufuncs)**, so they work efficiently on entire arrays.

Function	Description
sum	Sum of all elements or elements along an axis.
mean	Arithmetic mean.
std, var	Standard deviation and variance.
min, max	Minimum and maximum values.
argmin, argmax	Indices of the minimum and maximum elements.
cumsum	Cumulative sum of elements starting from 0.

Mathematical Functions in NumPy

These functions perform **element-wise mathematical operations**.

Common Mathematical Functions

◆ 1. Basic Arithmetic

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4])
```

```
print(np.add(a, 2)) # Addition  
print(np.subtract(a, 1)) # Subtraction  
print(np.multiply(a, 3)) # Multiplication  
print(np.divide(a, 2)) # Division
```

2. Statistical Functions in NumPy

These functions help in **data analysis and summarization**.

```
a = np.array([[1, 2, 3],  
[4, 5, 6]])  
print(np.mean(a))  
print(np.median(a))  
print(np.std(a))  
print(np.var(a))
```